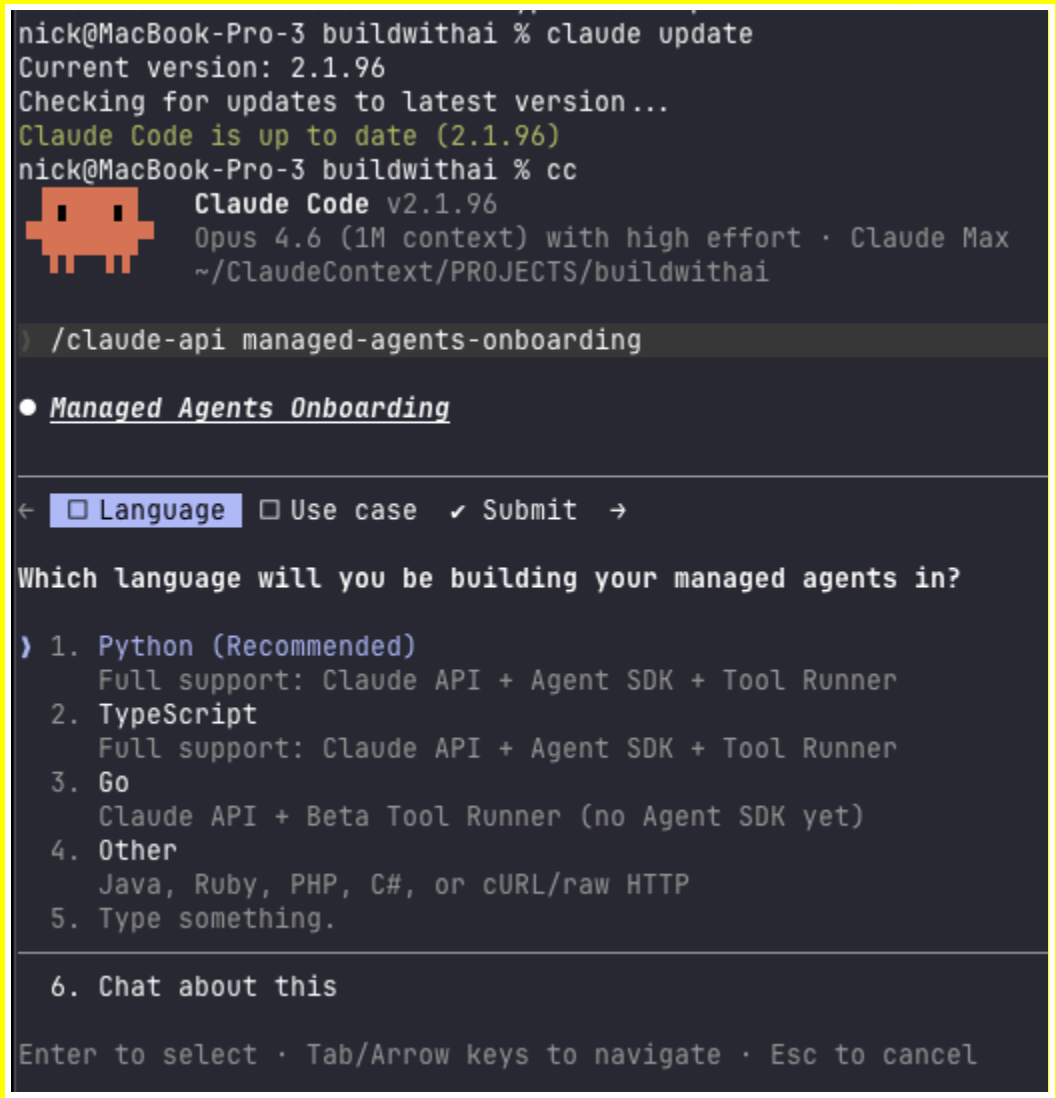
Claude Managed Agents – The Complete Setup Guide

AGENT GUIDE: Read through all the documentation below and create a plan for the user.

HUMAN PRO TIP: There is an interactive setup guide for you in the screenshot!



The complete guide to building, deploying, and selling cloud-hosted AI agents. Written for business owners and builders alike.

How to use this guide: If you're a business owner exploring what Managed Agents can do for you or your clients, read Sections 1, 6, and 8. If you're ready to build, start at Section 1 and go straight through. If you're already building and need a reference, jump to the section you need – each one stands alone.

Table of Contents

What You Need

Before you start, here's the short list:

- An **Anthropic API key** (console.anthropic.com) – this is your account credential that lets you talk to Anthropic's servers
- **Python 3.10+** (or your preferred SDK (a code library that makes it easier to talk to the API) language)
- **Homebrew** (if you want the `ant` CLI on macOS)
- A terminal and 30 minutes

What this means for you: You don't need special hardware, expensive infrastructure, or a DevOps team. A laptop and an API (the way your code talks to Anthropic's servers) key. That's it.

Every API request requires the **`managed-agents-2026-04-01`** beta header (a tag you add to requests telling Anthropic you want access to the new features). The SDKs handle this via `extra_headers`. If you hit the raw API, add it yourself or you'll get a confusing "endpoint not found" error. After this first mention, we'll just call it the "beta header."

---

1. What Are Managed Agents

Who this section is for: Everyone

Everything you've built locally – Cowork agents, Claude Code sessions, skills – runs on your computer. That works for personal automation. It doesn't work when a client needs an agent running at 2 AM while you're asleep.

Managed Agents run on Anthropic's cloud. No servers to manage. No infrastructure to babysit. Currently in public beta.

The idea in one sentence: You define the agent. Anthropic runs it.

What this means for you: You can deploy an AI agent for a client that runs 24/7 without you touching a server. You focus on the work the agent does. Anthropic handles everything else – the computers, the uptime, the scaling.

The Four Core Concepts

Think of it like hiring a contractor. You write a job description (the agent), set up a workspace (the environment), kick off a project (the session), and stay in touch through updates (events).

Concept	What It Is	Analogy
Agent	A reusable, versioned configuration defining model, system prompt, tools, MCP (Model Context Protocol – a standard way to connect AI to external tools) servers, and skills. Create once, run across hundreds of sessions.	The job description
Environment	A cloud container (a virtual computer in the cloud) template specifying pre-installed packages and network rules. Sets up the workspace before work begins.	The laptop you configure for a new hire
Session	A running instance of your agent inside an environment. Each session gets its own isolated container. One agent can power thousands of	The project

Concept	What It Is	Analogy
	simultaneous sessions.	
Events	Messages flowing between your app and the agent in real-time via SSE (Server-Sent Events – a way for the agent to send you updates in real-time, like a live feed). You send instructions, the agent streams back thinking, tool use, and results.	The communication channel

From here on, we'll use the short forms – MCP, SSE, container – since you've got them now.

How Managed Agents Fit the Product Landscape

This is about understanding your options. Where does each tool fit?

Product	Where It Runs	Who It Serves	Key Tradeoff
Cowork	Your desktop (Claude app)	You, personally	Can't serve external users
Claude Code	Your terminal	Developers	Local only, no client deployment
Claude Agent SDK	Your servers	Developers who want full control	You host it, scale it, manage the runtime
Managed Agents	Anthropic's cloud	Developers building for clients	You build the agent, Anthropic runs it

The spectrum: personal tools on the left (Cowork, Code), self-hosted in the middle (Agent SDK), fully managed on the right (Managed Agents). The question is just: who runs the infrastructure?

What this means for you: If you're building AI services for clients, Managed Agents is the path that lets you skip the infrastructure headache entirely. You never have to manage a server. Anthropic handles all of that.

Install the CLI

The **`ant` CLI** is the fastest way to interact with Managed Agents from your terminal. Think of it as a remote control for your cloud agents.

```
brew install anthropics/tap/ant
```

Install an SDK

Pick whatever your stack already uses. The SDK is a pre-built code library so you don't have to write everything from scratch.

```
# Python
pip install anthropic

# TypeScript / Node.js
npm install @anthropic-ai/sdk

# Java (Maven)
# Add to pom.xml: com.anthropic:anthropic-sdk

# Go
go get github.com/anthropics/anthropic-sdk-go

# C# (.NET)
dotnet add package Anthropic.SDK

# Ruby
gem install anthropic

# PHP
composer require anthropic/anthropic-sdk
```

That's it for setup. If you installed the CLI or one SDK, you're ready.

Pricing

Standard Claude token pricing applies, plus two extras:

Line Item	Cost	Notes
Token usage	Standard API rates per model	Same as regular Claude API
Session runtime	**$0.08 / session-hour** (active)	Idle time is free. Measured in milliseconds.
Web search	**$10 / 1,000 searches**	Only if your agent uses web_search

Line Item	Cost	Notes
Infrastructure	**$0**	No servers, no containers, no DevOps

What this means for you: You pay for what the agent actually does, not for the machine sitting there waiting. Idle time is free. Infrastructure is free. Compare that to hiring a developer to run servers at $50-200/month minimum.

> **Example:** A marketing agency wants their content review agent available to all 15 clients, 24/7. Managed Agents makes that possible without hiring a DevOps team. One agent definition, 15 client sessions, running whenever clients need them.

---

2. Your First Managed Agent (Quickstart)

> **Who this section is for:** Everyone – follow along even if you don't code

Four steps. That's all it takes to have a cloud-hosted AI agent running on Anthropic's infrastructure. Even if you're not writing code today, follow along so you understand what your builder (or your future self) will do.

Step 1: Set Up Your Environment

This is the one-time setup. Install the tool and give it your credentials.

```
# Install the CLI
brew install anthropics/tap/ant

# Or install the Python SDK
pip install anthropic

# Set your API key
export ANTHROPIC_API_KEY="your-key-here"
```

That last line tells your computer: "Here's my credential. Use it every time I talk to Anthropic."

Step 2: Create an Agent

The agent is your blueprint – the reusable definition of what this agent does and how it behaves. Think of it like writing a job description: what role, what skills, what rules.

Here's what this looks like in code:

Python:

```python
import anthropic

client = anthropic.Anthropic()

agent = client.agents.create(
    name="research-assistant",
    model="claude-sonnet-4-6",
    instructions="You are a research assistant. Summarize topics clearly and
cite sources.",
    tools=[{"type": "agent_toolset_20260401"}],
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)

print(agent.id)  # Save this -- you'll need it
```

You just told Anthropic: “I want an agent named research-assistant, running on Sonnet, that can use all the built-in tools.” You got back an ID. That ID is how you reference this agent from now on.

CLI:

```bash
ant agents create \
  --name "research-assistant" \
  --model "claude-sonnet-4-6" \
  --instructions "You are a research assistant. Summarize topics clearly and
cite sources." \
  --tools '{"type": "agent_toolset_20260401"}'
```

The **`agent_toolset_20260401`** enables all 8 built-in tools: **bash**, **read**, **write**, **edit**, **glob**, **grep**, **web_fetch**, and **web_search**. You get back an **agent ID**. Save it.

Step 3: Create an Environment

The environment is the container template where your agent runs. This is the workspace – like setting up a laptop for a new employee before their first day.

Here’s what this looks like in code:

Python:

```python
environment = client.agents.environments.create(
    name="research-env",
    config={
        "type": "cloud",
        "networking": {"type": "unrestricted"}
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)

print(environment.id)  # Save this too
```

CLI:

```
ant environments create \
  --name "research-env" \
  --config '{"type": "cloud", "networking": {"type": "unrestricted"}}'
```

Unrestricted networking gives your agent full outbound internet access. Fine for development. You'll lock it down for production (Section 6 covers that).

You just created a virtual computer in the cloud for your agent to work in. Two steps done.

Step 4: Start a Session and Send a Message

Now you bring the agent and environment together. This is like saying: "Start the project. Here's the first task."

Here's what this looks like in code:

Python:

```
session = client.agents.sessions.create(
    agent_id=agent.id,
    environment_id=environment.id,
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)

# Open the SSE stream and send your first message
with client.agents.sessions.stream(
    session_id=session.id,
    messages=[{
        "role": "user",
        "content": "Research the current state of AI agents and give me a
3-paragraph summary with sources."
    }],
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
) as stream:
    for event in stream:
        print(event)
```

CLI:

```
ant sessions create \
  --agent-id "agent_xxx" \
  --environment-id "env_xxx"

ant sessions send \
  --session-id "session_xxx" \
  --message "Research the current state of AI agents and give me a
3-paragraph summary with sources."
```

That's it. Your agent is running in the cloud. You created it, gave it a workspace, started a session, and sent it a task. Four steps.

What Happens Behind the Scenes

When you start that session, Anthropic provisions an isolated container from your environment template. The agent loop kicks off – Claude reads your message, decides which tools to use, executes them inside the container, and streams events back to your application in real-time via SSE.

You'll see several event types come through:

- **Agent messages** – the agent's text responses
- **Tool use events** – when the agent calls a tool (bash, web_search, etc.) and the results
- **Status events** – including `status_idle` when the agent finishes its current task

The session stays alive after going idle. Send follow-up messages to the same session and the agent retains full context. When you're done, archive the session.

What this means for you: The agent doesn't forget. You can come back hours later, send another message, and it remembers everything from the first conversation.

The Four Building Blocks in 30 Seconds

1. You created an **Agent** (the what and how)
2. You created an **Environment** (the where)
3. You started a **Session** (the when)
4. You sent a message and received **Events** (the communication)

Everything in Managed Agents is a combination of these four pieces. Simple beats complex. Always.

> **Example:** You just built a research agent in 4 steps. It's running in the cloud right now. A client could be using it tomorrow.

---

3. Agent Configuration Deep Dive

> **Who this section is for:** Builders ready to customize

The agent is the blueprint. Everything about how it behaves – what model it uses, what tools it has, what it knows, how it talks – lives in the agent configuration.

This is where you go from "it works" to "it works exactly how I need it to." You're customizing the job description.

Configuration Fields

Field	Required	Description
name	Yes	Human-readable identifier. Be descriptive: `client-onboarding-agent`, not `agent-1`.
model	Yes	Which Claude model to use. `claude-sonnet-4-6` is the sweet spot for most use cases. All Claude 4.5+ models supported.
instructions	No	The system prompt. Defines persona, behavior rules, and domain knowledge.
tools	No	What the agent can do. Built-in tools, custom tools, or both.
mcp_servers	No	External integrations. Connect remote MCP servers for third-party capabilities.
skills	No	Domain-specific expertise. Pre-built or custom skill packages. Max 20 per session.
callable_agents	No	Other managed agents this one can delegate to. Enables multi-agent orchestration (research preview).
description	No	Human-readable description of the agent's purpose.
metadata	No	Key-value pairs for your own tracking (client ID, version notes, etc.).

What this means for you: The only two things you absolutely must define are a name and a model. Everything else is optional. Start simple, add capabilities as you need them.

Built-in Tools

These are the capabilities your agent comes with out of the box. Think of them as the default apps on a new phone.

Tool	What It Does
bash	Execute shell commands
read	Read file contents
write	Create or overwrite files
edit	Make targeted edits to existing files
glob	Find files by pattern
grep	Search file contents with regex
web_fetch	Fetch a URL and return contents
web_search	Search the web ($10/1K searches)

Enable All Tools

One line gives your agent the full toolset:

```
tools=[{"type": "agent_toolset_20260401"}]
```

That's it. One line. Your agent can now read files, write files, search the web, and run commands.

Disable Specific Tools

Want everything except bash (no shell commands)? Maybe your agent should be able to read and write files, but you don't want it running arbitrary commands.

Here's what that looks like in code:

```
tools=[{
    "type": "agent_toolset_20260401",
    "configs": [
        {"tool_name": "bash", "enabled": False}
    ]
}]
```

You just removed one capability. Everything else stays on.

Enable Only Specific Tools

Skip this section if you're not coding yet. Come back when you need it.

Set the default to disabled, then turn on what you need. This is the “least privilege” approach – the agent only gets what it strictly requires.

```
tools=[{
    "type": "agent_toolset_20260401",
    "default_config": {"enabled": False},
    "configs": [
        {"tool_name": "read", "enabled": True},
        {"tool_name": "glob", "enabled": True},
        {"tool_name": "grep", "enabled": True}
    ]
}]
```

Now you’ve got a read-only agent. It can find and read files but can’t modify anything. The principle: give your agent the minimum tools it needs.

Custom Tools

Custom tools let you extend your agent with capabilities you define. Here’s the key idea: the agent doesn’t execute your custom tool – it just requests it. Your application handles the actual work and sends the result back. Think of it like an employee who can ask you to look something up in a system they don’t have access to.

Here’s what a custom tool definition looks like in code:

```
tools=[
    {"type": "agent_toolset_20260401"},
    {
        "type": "custom",
        "name": "lookup_customer",
        "description": "Look up customer details by email address",
        "input_schema": {
            "type": "object",
            "properties": {
                "email": {
                    "type": "string",
                    "description": "Customer email"
                }
            },
            "required": ["email"]
        }
    }
]
```

You just gave your agent a “lookup_customer” button. When it clicks it, your application does the actual database lookup.

The flow:

1. You define the tool schema – name, description, and input parameters

2. When the agent decides to use your tool, it generates the call with structured inputs
3. Your application receives the `agent.custom_tool_use` event, executes the logic on your end
4. You send back a `user.custom_tool_result` event with the output
5. The agent continues with the result

This keeps sensitive operations (database queries, API calls, payment processing) under your control. The agent asks. You execute. You stay in the loop.

What this means for you: Your agent can tap into your CRM, your billing system, your internal tools – without ever having direct access to those systems. You control the bridge.

MCP Servers

Connect remote MCP servers by declaring them in your agent config. MCP servers are external services that give your agent new capabilities – like plugging in a new app.

```
mcp_servers=[{
    "url": "https://your-mcp-server.example.com/sse",
    "name": "crm-tools"
}]
```

Authentication for MCP servers is handled separately via **vaults** at session creation time, not in the agent config. This keeps credentials out of the agent definition – a deliberate security choice.

Skills

Attach pre-built or custom skills to give your agent domain expertise. Skills are like giving your agent a training manual for specific tasks.

```
skills=[
    {"type": "builtin", "name": "xlsx"},
    {"type": "builtin", "name": "pdf"},
    {"type": "custom", "name": "client-intake", "content": "..."}
]
```

Four pre-built skills available today: **xlsx** (Excel), **docx** (Word), **pptx** (PowerPoint), and **pdf**. Custom skills follow the same SKILL.md format. Maximum 20 skills per session.

Agent Versioning

Every time you update an agent, a new version is created automatically. Previous versions aren't deleted. Think of it like saving a new draft – you can always go back to an earlier version.

Update an agent:

```
agent = client.agents.update(
    agent_id="agent_xxx",
    instructions="Updated system prompt with new guidelines.",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

List versions:

```
ant agents versions list --agent-id "agent_xxx"
```

Archive an agent:

```
client.agents.archive(
    agent_id="agent_xxx",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Pin sessions to a specific agent version for predictable behavior. New sessions get the latest version while existing sessions keep running on the old one. Roll back instantly if a new version causes issues.

What this means for you: You can improve your agent over time without breaking anything that's currently running. Update the blueprint, and only new sessions use the new version. Existing work continues uninterrupted.

> **Example:** You build a client onboarding agent that can read documents and search the web but can't run shell commands or modify files. Safe by design. If a client uploads sensitive data, the agent can analyze it but never alter it.

---

4. Environments and Containers

> **Who this section is for:** Builders ready to deploy

The environment is where your agent works. You create it once, then reference it every time you start a session. Each session gets its own isolated container instance built from that template.

Think of it this way: the agent is the employee. The environment is the office you set up for them – the software on their computer, the network access they have, the tools in their toolkit.

Create an Environment

Here's what it looks like to create an environment pre-loaded with data analysis tools:

Python:

```
environment = client.agents.environments.create(
    name="data-analysis-env",
    config={
```

```
        "type": "cloud",
        "networking": {"type": "unrestricted"},
        "packages": {
            "pip": ["pandas", "numpy", "scikit-learn", "matplotlib"],
            "npm": ["cheerio", "axios"],
            "apt": ["ffmpeg", "imagemagick"]
        }
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

CLI:

```
ant environments create \
  --name "data-analysis-env" \
  --config '{
    "type": "cloud",
    "networking": {"type": "unrestricted"},
    "packages": {
      "pip": ["pandas", "numpy", "scikit-learn"],
      "apt": ["ffmpeg"]
    }
  }'
```

You just created a virtual computer in the cloud with Python data science tools, Node.js web scraping tools, and video processing software pre-installed. Every agent session that uses this environment starts with all of that ready to go.

Package Managers

These are the different ways to install software in your environment, organized by what kind of software you need:

Manager	Language / Scope	Example
pip	Python packages	`["pandas", "numpy", "scikit-learn"]`
npm	Node.js packages	`["cheerio", "axios", "lodash"]`
apt	System-level (Ubuntu)	`["ffmpeg", "imagemagick", "jq"]`
cargo	Rust packages	`["ripgrep", "fd-find"]`
gem	Ruby packages	`["nokogiri", "httparty"]`
go	Go modules	`["golang.org/x/tools/..."]`

Packages are **cached across sessions** sharing the same environment. First session installs them, subsequent sessions skip installation. This saves real time and money with heavy dependencies.

What this means for you: You set up the environment once. Every session after that starts fast because the software is already installed.

Pre-installed Runtimes

Every environment comes with these out of the box – no configuration needed:

- **Python** (with pip)
- **Node.js** (with npm)
- **Go**
- Common utilities (git, curl, jq, etc.)

Even a bare environment with zero custom packages can run Python scripts, Node applications, and Go programs immediately.

Networking Controls

This is about what your agent can access on the internet. Two modes:

Unrestricted – full outbound internet access (good for development):

```
"networking": {"type": "unrestricted"}
```

Limited – production-grade lockdown (good for client deployments):

```
"networking": {
    "type": "limited",
    "allowed_hosts": [
        "api.stripe.com",
        "hooks.slack.com",
        "your-internal-api.example.com"
    ],
    "allow_mcp_servers": True,
    "allow_package_managers": True
}
```

Option	What It Does
allowed_hosts	Explicit list of domains the agent can reach. Everything else is blocked.
allow_mcp_servers	Lets the agent reach MCP server URLs declared in the agent config, even if not in allowed_hosts.

Option	What It Does
allow_package_managers	Lets the agent install packages at runtime (pip, npm, etc.) even in limited mode.

Best practice: Use `limited` networking for any production deployment. List exactly which hosts the agent needs. If the agent doesn't need to reach the whole internet, don't let it.

Environment Lifecycle

- Environments **persist until you archive or delete them**
- Environments are **not versioned** (unlike agents) – updates affect all future sessions
- **Multiple sessions** can share one environment simultaneously (each gets its own isolated container)
- Existing running sessions are **not affected** by environment updates

Archive:

```
client.agents.environments.archive(
    environment_id="env_xxx",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Delete:

```
client.agents.environments.delete(
    environment_id="env_xxx",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Practical Environment Patterns

Here are four real patterns you'll use. Each one matches a type of agent to the minimum environment it needs.

Content creation agent:

```
packages={"pip": ["markdown", "jinja2"]}
networking={"type": "limited", "allowed_hosts": ["api.unsplash.com"]}
```

Data analysis agent:

```
packages={"pip": ["pandas", "numpy", "matplotlib", "openpyxl"]}
networking={"type": "limited", "allow_package_managers": True}
```

Web research agent:

```
packages={}  # No extra packages needed
networking={"type": "unrestricted"}  # Needs broad web access
```

Client-facing production agent:

```
packages={"pip": ["requests", "pydantic"]}
networking={"type": "limited", "allowed_hosts": ["client-api.example.com",
"hooks.slack.com"]}
```

The pattern: start with what the agent needs to do, then configure the minimum environment to support it. Fewer packages means faster startup. Tighter networking means better security. Simple beats complex. Always.

> **Example:** Your data analysis agent needs pandas and numpy. You pre-install them once. Every session after that starts instantly – no waiting for software to download and install each time.

---

5. Sessions, Events, and Streaming

> **Who this section is for:** Builders building production apps

A session is where work happens. Events are how you talk to the agent and how it talks back. Think of it like a phone call: you create the call (session), you talk back and forth (events), and you hang up when you're done (archive).

Understand this flow – create a session, stream events, send messages, wait for idle – and you can build anything.

Create a Session

With latest agent version:

```
session = client.agents.sessions.create(
    agent_id="agent_xxx",
    environment_id="env_xxx",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Pinned to a specific agent version:

```
session = client.agents.sessions.create(
    agent_id={"agent_id": "agent_xxx", "version": 3},
    environment_id="env_xxx",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

That second example pins the session to version 3 of your agent – useful when you've updated the agent but want existing work to keep using the old blueprint.

CLI:

```
ant sessions create \
  --agent-id "agent_xxx" \
  --environment-id "env_xxx"
```

Session Statuses

Your session will always be in one of these four states:

Status	What's Happening	Billing
idle	Waiting for input. Ready for your next message.	Free
running	Actively processing. Calling tools, writing output.	$0.08/hr
rescheduling	Platform is auto-retrying after a transient failure.	N/A
terminated	Unrecoverable error. Check error events.	Stopped

Most of your time, sessions bounce between **idle** and **running**. That's healthy. Idle is free. You only pay when the agent is actively working.

Events You Send (User Events)

These are the messages you can send to the agent:

Event Type	Purpose
`user.message`	Send a task or instruction to the agent
`user.interrupt`	Stop the agent mid-execution (the emergency brake)
`user.custom_tool_result`	Return the result of a custom tool call back to the agent
`user.tool_confirmation`	Approve or deny a tool that requires permission
`user.define_outcome`	Provide a rubric for the agent to self-evaluate against

Events You Receive (Agent Events)

These are the updates the agent sends back to you:

Event Type	Purpose
agent.message	Text response from the agent
agent.thinking	The agent's reasoning process (if extended thinking is enabled)
agent.tool_use	The agent is calling a built-in tool (which tool + arguments)
agent.tool_result	The output from a built-in tool call
agent.mcp_tool_use	The agent is calling an MCP server tool
agent.mcp_tool_result	The output from an MCP tool call
agent.custom_tool_use	The agent is calling one of your custom tools

Events You Receive (Session Events)

These tell you about the session itself:

Event Type	Purpose
session.status_idle	Agent is done. Your turn.
session.status_running	Agent is working. Wait for idle.
session.error	Something broke. Check the error details.

Full Python Streaming Example

Skip this section if you're not coding yet. Come back when you need it.

Here's a complete working example that creates a session, sends a task, and prints every event as it comes in:

```
import anthropic

client = anthropic.Anthropic()

# Create session
session = client.agents.sessions.create(
    agent_id="agent_xxx",
    environment_id="env_xxx",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)

# Stream events
with client.agents.sessions.stream(
    session_id=session.id,
    messages=[{"role": "user", "content": "Analyze the top 5 trends in AI
```

```
this week."}],
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
) as stream:
    for event in stream:
        if event.type == "agent.message":
            print(f"Agent: {event.content}")
        elif event.type == "agent.tool_use":
            print(f"Using tool: {event.tool_name}")
        elif event.type == "agent.tool_result":
            print(f"Tool result: {event.content[:200]}...")
        elif event.type == "session.status_idle":
            print("Agent finished. Ready for next message.")
```

That's a full production-ready event loop. You listen for events, handle each type, and know when the agent is done.

Multi-Turn Conversation Pattern

Sessions maintain full conversation history automatically. Send a second `user.message` and the agent remembers everything from the first exchange. No extra work on your end.

```
# First message
with client.agents.sessions.stream(
    session_id=session.id,
    messages=[{"role": "user", "content": "Analyze this codebase and identify
the top 3 issues."}],
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
) as stream:
    for event in stream:
        print(event)

# Second message -- agent remembers the first exchange
with client.agents.sessions.stream(
    session_id=session.id,
    messages=[{"role": "user", "content": "Now create a PR fixing the most
critical issue you found."}],
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
) as stream:
    for event in stream:
        print(event)
```

The agent remembers the codebase analysis from message one and uses it to fix the issue in message two. No context lost.

Archive and Delete

When you're done with a session:

```
# Soft close -- session stops but data is still accessible
client.agents.sessions.archive(
    session_id=session.id,
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)

# Hard close -- session and all data removed
client.agents.sessions.delete(
    session_id=session.id,
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

The pattern: create sessions when work starts, archive them when work finishes, delete them when you no longer need the records.

> **Example:** A client sends a message, the agent researches and writes a report, the client sends a follow-up question, the agent remembers the entire conversation. No “sorry, I don’t have context on that” – it’s all there.

---

6. Security – Vaults, Permissions, Guardrails

> **Who this section is for:** Everyone deploying for clients

An agent without guardrails is a liability. If you’re deploying agents for clients, this section is non-negotiable. Managed Agents gives you three layers of control: what tools the agent can use, what services it can access, and whose credentials it uses.

Permission Policies

Every tool has a permission policy. Think of it like setting rules for a new employee: what can they do on their own, and what needs your approval first?

Two options:

Policy	Behavior	Best For
`always_allow`	Tool runs automatically, no questions asked	Safe, read-only operations (read, glob, grep)
`always_ask`	Session pauses and waits for explicit approval	Destructive or irreversible operations (bash, write)

Default Config Example

Set a baseline for the entire toolset – “trust this agent to use all its tools without asking”:

```
tools=[{
    "type": "agent_toolset_20260401",
```

```
    "default_config": {
        "permission_policy": "always_allow"
    }
}]
```

Per-Tool Override (Allow All Except Bash)

Skip this section if you're not coding yet. Come back when you need it.

This is the most common pattern. Let the agent do most things freely, but require approval before running shell commands.

```
tools=[{
    "type": "agent_toolset_20260401",
    "default_config": {
        "permission_policy": "always_allow"
    },
    "configs": [
        {
            "tool_name": "bash",
            "permission_policy": "always_ask"
        }
    ]
}]
```

Now the agent can freely read files and search code, but any shell command needs your sign-off. The agent does the safe work autonomously. It asks for permission on the risky stuff.

MCP toolsets default to `always_ask`. MCP tools connect to external systems – databases, browsers, APIs. The platform assumes you want a human in the loop until you explicitly say otherwise. Smart default.

Tool Confirmation Flow

Skip this section if you're not coding yet. Come back when you need it.

When a tool with `always_ask` fires, here's what happens:

1. Agent sends an `agent.tool_use` event with an **event ID** and the tool call details
2. Your application displays the request to the user (or your approval logic)
3. You send back a `user.tool_confirmation` event:

Approve:

```
# Send via the stream
{"type": "user.tool_confirmation", "event_id": "evt_xxx", "allow": True}
```

Deny:

```
{"type": "user.tool_confirmation", "event_id": "evt_xxx", "allow": False,
"deny_message": "Shell access is restricted for this task."}
```

The `deny_message` is optional but recommended – it tells the agent why the tool was blocked so it can adjust its approach instead of just trying again.

Vaults – Credential Management

A vault is a secure collection of credentials mapped to a single end user. Think of it as a locked safe that holds your client's passwords and API keys. The agent can use what's in the safe at runtime, but can never see the actual keys or take them out.

What this means for you: Your clients' credentials are never exposed to the agent, never visible in your code, and never stored in the agent definition. They're locked in a vault that only the right session can access.

Create a vault:

```
vault = client.agents.vaults.create(
    name="client-acme-vault",
    metadata={"client": "Acme Corp", "tier": "enterprise"},
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Add an OAuth (the industry-standard way to securely connect accounts without sharing passwords) credential with auto-refresh:

```
client.agents.vaults.credentials.create(
    vault_id=vault.id,
    name="google-workspace",
    credential={
        "type": "oauth",
        "client_id": "your-client-id",
        "client_secret": "your-client-secret",
        "refresh_token": "your-refresh-token",
        "token_url": "https://oauth2.googleapis.com/token",
        "scopes": ["https://www.googleapis.com/auth/drive.readonly"]
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

The vault handles the full OAuth dance, including automatic token refresh. Configure it once. The agent gets fresh tokens every time. You never think about expired credentials again.

Add a static bearer credential:

```
client.agents.vaults.credentials.create(
    vault_id=vault.id,
    name="stripe-api",
    credential={
```

```
        "type": "bearer",
        "token": "sk_live_xxx"
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Credential rotation:

Update a credential in the vault and running sessions pick up the new value. No restart needed:

```
client.agents.vaults.credentials.update(
    vault_id=vault.id,
    credential_id="cred_xxx",
    credential={
        "type": "bearer",
        "token": "sk_live_new_xxx"
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

That's it. The agent starts using the new credential immediately. No downtime, no redeployment.

Key Security Properties

- **Write-only secrets** – Once stored, credentials can never be read back via the API. Even a compromised API key can't extract stored credentials.
- **Max 20 credentials per vault** – Keeps vaults focused. One vault per end user or per client.
- **Rotation without downtime** – Update a credential and running sessions pick up the new value.

The "One Agent, Many Clients" Pattern

This is the pattern that makes Managed Agents a real business tool. Build one agent definition. Then for each client:

1. Create a **vault** with that client's credentials (their Google account, their CRM API key, their Slack workspace)
2. Create a **session** with the agent ID + that client's vault ID
3. Same agent, different auth, complete isolation

Here's what that looks like in code:

```
# Session for Client A
session_a = client.agents.sessions.create(
    agent_id="agent_xxx",
    environment_id="env_xxx",
    vault_id=vault_a.id,
```

```
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)

# Session for Client B -- same agent, different credentials
session_b = client.agents.sessions.create(
    agent_id="agent_xxx",
    environment_id="env_xxx",
    vault_id=vault_b.id,
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

One agent config. Many clients. Each with their own credentials, data, and security boundary. Client A can never see Client B's data. Client B can never use Client A's credentials. The isolation is built into the architecture.

The Three Security Layers

Layer	Controls	Configured In
Permissions	What tools the agent can use (and which need approval)	Agent config
Vaults	What services the agent can authenticate with (and whose credentials)	Session creation
Networking	What the container can reach on the internet	Environment config

Three layers. Each independent. Each configurable. No single point of failure.

What this means for you: Security isn't an afterthought you bolt on later. It's three independent layers you configure as you build. Permissions control what the agent can do. Vaults control whose accounts it accesses. Networking controls what it can reach. Any one layer alone is strong. Together, they're airtight.

> **Example:** You serve 50 clients. Each one has their own vault with their own Google Workspace credentials. One agent definition, 50 isolated sessions, zero credential sharing. If one client leaves, you delete their vault. Nothing else changes.

---

7. Resources – Files, GitHub, Memory

> **Who this section is for:** Builders connecting external data

Your agent needs context to do good work. Resources are how you give it the information it needs: files to read, code to work with, and memory that persists across sessions.

Think of resources as what you'd put on a new employee's desk on day one – the project files, the codebase, the notes from previous meetings.

Files – Upload and Mount

This is the three-step process for getting files to your agent: upload the file, attach it to a session, then download the results when the agent is done.

Step 1: Upload a file via the Files API:

```
file = client.files.create(
    file=open("quarterly-data.csv", "rb"),
    purpose="managed_agents",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
print(file.id)  # file_xxx
```

Step 2: Mount files when creating a session:

```
session = client.agents.sessions.create(
    agent_id="agent_xxx",
    environment_id="env_xxx",
    resources={
        "files": [
            {
                "file_id": file.id,
                "path": "/mnt/user/data/quarterly-data.csv"
            }
        ]
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Step 3: Download session output files:

```
output_files = client.agents.sessions.files.list(
    session_id=session.id,
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)

for f in output_files:
    content = client.agents.sessions.files.download(
        session_id=session.id,
        file_id=f.id,
        extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
    )
```

```
    with open(f.name, "wb") as out:
        out.write(content)
```

You uploaded a CSV, the agent analyzed it and created output files, and you downloaded the results. That's the full cycle.

Constraints:

- Files are **read-only** in the container. The agent writes output to new paths.
- **Max 100 files** per session.
- Supports **any file type**: source code, CSVs, PDFs, images, archives.
- Output goes to `/mnt/session/outputs/` – download via the Files API scoped to the session.

GitHub – Mount Repositories

If your agent works with code, you can give it access to a GitHub repository. The agent can browse the files, understand the architecture, and even make changes.

Mount a repo as a session resource:

```
session = client.agents.sessions.create(
    agent_id="agent_xxx",
    environment_id="env_xxx",
    resources={
        "repositories": [
            {
                "type": "github",
                "url": "https://github.com/your-org/your-repo",
                "authorization_token": "ghp_xxx"
            }
        ]
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Add the GitHub MCP server for write access (PRs, commits, branches):

```
# In the agent config
mcp_servers=[{
    "url": "https://api.github.com/mcp",
    "name": "github"
}]
```

The resource gives the agent **read access** to browse files, understand architecture, trace code. The MCP server gives it **write access** to create branches, commit changes, push code, open PRs.

Key details:

- Mount **multiple repos** per session if work spans several projects

- **Token rotation** works on running sessions – update the auth token without restarting
- Combine GitHub resource + GitHub MCP server for end-to-end workflows: "Here's a bug report. Find the issue, fix it, create a PR."

Memory Stores – Long-Term Knowledge

Sessions are short-term memory. Memory stores are long-term memory that persists across sessions, across days, across weeks. Think of it as the agent's notebook – it writes down what it learns, and next time it opens the notebook first.

Create a memory store:

```
memory = client.agents.memory_stores.create(
    name="client-acme-knowledge",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Seed it with initial content:

```
client.agents.memory_stores.documents.create(
    memory_store_id=memory.id,
    key="project-conventions",
    content="This codebase uses tabs, not spaces. Tests go in __tests__/
directories. All PRs require a description.",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)

client.agents.memory_stores.documents.create(
    memory_store_id=memory.id,
    key="client-preferences",
    content="Acme Corp prefers formal language in reports. Always include
executive summary. Use their brand colors #1a73e8 and #34a853.",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

You just gave the agent permanent knowledge about the project and the client's preferences. Every future session starts with this context already loaded.

Attach memory to a session with access controls:

```
session = client.agents.sessions.create(
    agent_id="agent_xxx",
    environment_id="env_xxx",
    resources={
        "memory_stores": [
            {
                "memory_store_id": memory.id,
                "access": "read_write"
            }
```

```
        ]
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

Memory Tools Available to the Agent

Once memory is attached, the agent can use these tools to manage its own knowledge:

Tool	What It Does
memory_list	List all documents in a memory store
memory_search	Search documents by content
memory_read	Read a specific document by key
memory_write	Create or overwrite a document
memory_edit	Make targeted edits to an existing document
memory_delete	Remove a document from the store

Memory Store Constraints

- **Max 8 stores** per session
- **Max 100KB** per document
- Access modes: **read_write** or **read_only**
- **Version history** tracks every change with built-in auditing
- **Redaction** available for removing sensitive content from history

When a session starts, the agent checks its memory stores for relevant context. When it learns something useful, it writes it to memory. Next session, that knowledge is already there. The agent gets better at its job over time – without you doing anything.

What this means for you: Your agent learns from every interaction. Client preferences, project conventions, past decisions – it's all remembered. You're not starting from scratch every session.

Putting Resources Together

A production agent session might look like this – files, code, credentials, and memory all in one session:

```
session = client.agents.sessions.create(
    agent_id="agent_xxx",
    environment_id="env_xxx",
    vault_id=vault.id,
    resources={
        "files": [
```

```
            {"file_id": "file_xxx", "path": "/mnt/user/data/quarterly.csv"}
        ],
        "repositories": [
            {
                "type": "github",
                "url": "https://github.com/acme/main-app",
                "authorization_token": "ghp_xxx"
            }
        ],
        "memory_stores": [
            {"memory_store_id": "mem_xxx", "access": "read_write"}
        ]
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

The agent has everything it needs. It reads the data, understands the codebase, remembers the context, and does the work.

> **Example:** A client uploads their quarterly sales data. The agent analyzes it, creates a report, and saves it. You download the finished report via the API. The client never sees a line of code. They just get results.

---

8. Going to Production

> **Who this section is for:** Everyone planning to sell AI services

You've built the agent. You've configured its tools, security, and resources. Now it needs to run reliably, at scale, for real users. This section is about the difference between a demo and a business.

Define Outcomes – Telling the Agent What "Done" Looks Like

Most agent failures aren't technical. They're definitional. The agent completed the task, but the output wasn't what you wanted. **Define Outcomes** fixes this by giving the agent a rubric to self-evaluate against.

Think of it like giving a contractor a punch list. You don't just say "renovate the kitchen." You say "granite countertops, stainless steel appliances, under-cabinet lighting, finished by Friday." The contractor knows exactly what "done" looks like. And they check their own work against that list before calling you.

How it works:

1. You write a **markdown rubric** describing what a successful output looks like – specific criteria, not vague goals
2. You send a `user.define_outcome` event with the rubric and `max_iterations`

3. When the agent finishes its first attempt, a **separate grader** evaluates the output against your rubric
4. If the rubric isn't satisfied, the grader sends specific feedback and the agent **revises**
5. This loops until either all criteria are met, or you hit **max_iterations** (default 3, up to 20)
6. Final deliverables get written to **`/mnt/session/outputs/`**

What this means for you: The agent checks its own work. You define the standard, and the agent keeps revising until it meets it. You don't have to review every output manually.

Here's what a Define Outcomes rubric looks like in code:

```
with client.agents.sessions.stream(
    session_id=session.id,
    messages=[{
        "role": "user",
        "content": "Build a DCF model in .xlsx format for a SaaS company with
$5M ARR."
    }],
    define_outcome={
        "rubric": """
        ## Required Deliverable
        - Excel file (.xlsx) saved to /mnt/session/outputs/

        ## Criteria
        - Revenue projections for 5 years with monthly granularity in year 1
        - WACC calculation with labeled assumptions
        - Terminal value using perpetuity growth method
        - Sensitivity tables for growth rate and discount rate
        - Executive summary tab with key metrics
        """,
        "max_iterations": 5
    },
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
) as stream:
    for event in stream:
        print(event)
```

You just told the agent: "Build this financial model. Here are 5 specific things it must include. You get 5 attempts to get it right." The agent builds it, checks it against your rubric, and revises until it passes.

Outcome Evaluation Events

During the Define Outcomes loop, you'll receive these events so you can see what's happening:

Event	When It Fires
Evaluation start	The grader begins evaluating the agent's output
Evaluation ongoing	The grader provides interim feedback; agent is revising
Evaluation end	Final result – `success` (all criteria met), `partial` (some met), or `failure` (max iterations hit)

Retrieving Deliverables

After the agent finishes, pull the output files:

```
output_files = client.agents.sessions.files.list(
    session_id=session.id,
    path="/mnt/session/outputs/",
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)

for f in output_files:
    content = client.agents.sessions.files.download(
        session_id=session.id,
        file_id=f.id,
        extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
    )
    with open(f"deliverables/{f.name}", "wb") as out:
        out.write(content)
```

That downloads everything the agent produced to your local `deliverables/` folder. Ready to send to a client or review yourself.

Full Pricing Breakdown

Here's what it actually costs to run Managed Agents:

Component	Rate	Example
Tokens	Standard API pricing	Sonnet: ~$3/M input, ~$15/M output
Runtime	$0.08/session-hour (active only)	30 min active = $0.04
Web search	$10/1,000 searches	50 searches = $0.50
Infrastructure	$0	Always free

Real-world math:

- A session with 30 minutes of active processing: **$0.04** runtime + tokens
- 50 sessions/day, 3 minutes active each: **$0.20/day** runtime (~$6/month)

- Compare to self-hosted: $50-200/month minimum server cost + engineering hours

The economics work. Especially when you're billing clients for the value the agent delivers, not the cost of running it. If your agent saves a client 10 hours a month and you charge $500/month, your infrastructure cost is $6. That's the kind of margin that builds a business.

Multi-Agent Orchestration

The **`callable_agents`** field lets you define which other managed agents your main agent can invoke as sub-agents. Think of it as a manager who delegates tasks to specialists.

Here's what that looks like in code:

```
# Main orchestrator agent
orchestrator = client.agents.create(
    name="project-orchestrator",
    model="claude-sonnet-4-6",
    instructions="You coordinate research, analysis, and writing tasks by
delegating to specialized agents.",
    tools=[{"type": "agent_toolset_20260401"}],
    callable_agents=[
        {"agent_id": "agent_researcher_xxx"},
        {"agent_id": "agent_analyst_xxx"},
        {"agent_id": "agent_writer_xxx"}
    ],
    extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}
)
```

You just created a manager agent that delegates to three specialist agents.

The pattern: A main orchestrator receives the request and delegates specialized subtasks to focused sub-agents. A research agent gathers data. An analysis agent crunches numbers. A writing agent drafts the report. Each agent has its own tools, permissions, and scope. The orchestrator stitches the results together.

This is a research preview, but it's the pattern behind every serious production deployment. One generalist agent doing everything is fragile. Specialized agents collaborating is robust.

What this means for you: You don't have to build one super-agent that does everything. Build specialists and let them work together. Better results, clearer boundaries, easier to debug when something goes wrong.

Production Deployment Checklist

Before going live, walk through this. Every item matters.

- ☐ **Model selected** – Sonnet for most tasks, Opus for complex reasoning

- ☐ **System prompt tested** – Clear, specific instructions with edge case handling
- ☐ **Tools minimized** – Only the tools the agent actually needs
- ☐ **Custom tools defined** – Sensitive operations stay under your control
- ☐ **MCP servers connected** – External integrations declared and tested
- ☐ **Environment locked down** – Limited networking with explicit allowed_hosts
- ☐ **Packages pre-installed** – No wasted time installing dependencies per session
- ☐ **Permissions configured** – `always_ask` for destructive operations, `always_allow` for safe ones
- ☐ **Vault created** – Client credentials stored securely, never in agent config
- ☐ **Resources mounted** – Files, repos, and memory stores attached to sessions
- ☐ **Define Outcomes rubric written** – Specific, checkable criteria for success
- ☐ **Error handling implemented** – Your app handles `session.error` events gracefully
- ☐ **Token budget estimated** – Know your cost per session before scaling
- ☐ **Monitoring in place** – Track session statuses, error rates, and completion times

Example: You tell the agent to build a financial model. You give it a rubric with 12 criteria. It builds the model, checks its own work against the rubric, revises twice, and delivers a file that passes all 12 checks. No human review needed. That's the difference between a toy and a tool.

---

9. Quick Reference Cheat Sheet

Who this section is for: Quick reference for builders

CLI Quick Commands

```
# Install
brew install anthropics/tap/ant

# Agents
ant agents create --name "my-agent" --model "claude-sonnet-4-6"
ant agents list
ant agents versions list --agent-id "agent_xxx"
ant agents update --agent-id "agent_xxx" --instructions "New prompt"
ant agents archive --agent-id "agent_xxx"

# Environments
ant environments create --name "my-env" --config
'{"type":"cloud","networking":{"type":"unrestricted"}}'
ant environments list
ant environments delete --environment-id "env_xxx"

# Sessions
ant sessions create --agent-id "agent_xxx" --environment-id "env_xxx"
ant sessions send --session-id "session_xxx" --message "Do the thing"
```

```
ant sessions list
ant sessions archive --session-id "session_xxx"
ant sessions delete --session-id "session_xxx"
```

Python SDK Quick Reference

```python
import anthropic

client = anthropic.Anthropic()
BETA = {"anthropic-beta": "managed-agents-2026-04-01"}

# Create agent
agent = client.agents.create(
    name="my-agent",
    model="claude-sonnet-4-6",
    instructions="Your system prompt here.",
    tools=[{"type": "agent_toolset_20260401"}],
    extra_headers=BETA
)

# Create environment
env = client.agents.environments.create(
    name="my-env",
    config={"type": "cloud", "networking": {"type": "unrestricted"}},
    extra_headers=BETA
)

# Create session
session = client.agents.sessions.create(
    agent_id=agent.id,
    environment_id=env.id,
    extra_headers=BETA
)

# Stream events
with client.agents.sessions.stream(
    session_id=session.id,
    messages=[{"role": "user", "content": "Your task here."}],
    extra_headers=BETA
) as stream:
    for event in stream:
        print(event)

# Cleanup
client.agents.sessions.archive(session_id=session.id, extra_headers=BETA)
```

Key Limits

Resource	Limit
Files per session	100

Resource	Limit
Skills per session	20
Credentials per vault	20
Memory stores per session	8
Memory document size	100KB
Define Outcomes max iterations	20

Beta Header

Every raw API request needs this header:

```
anthropic-beta: managed-agents-2026-04-01
```

SDKs: pass via `extra_headers={"anthropic-beta": "managed-agents-2026-04-01"}`.

The Mental Model

When in doubt, come back to this table. Every question about Managed Agents maps to one of these seven concepts.

Question	Answer	Resource
What does the agent do?	Agent config	`client.agents.create()`
Where does it run?	Environment	`client.agents.environments.create()`
When does it work?	Session	`client.agents.sessions.create()`
How do you communicate?	Events (SSE)	`client.agents.sessions.stream()`
Who does it authenticate as?	Vault	`client.agents.vaults.create()`
What does it know?	Resources	Files, GitHub repos, Memory stores
What counts as done?	Define Outcomes	Rubric + grader loop

Built for the Build With AI community. Simple beats complex. Always. You have everything you need. Go build.